

Saparali Adina

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.

Deliverable: Run it 5 times. Record the value of π .

- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

Run	Total Hits	Approximate π
1	16,162,139	1.29297112
2	22,748,344	1.81986752
3	12,552,043	1.00416344
4	12,074,923	0.96599384
5	13,056,509	1.04452072

Part 1 Result: I ran the program 5 times and received different values of π each time. The results were 1.2930, 1.8199, 1.0042, 0.9660 and 1.0445. The results are incorrect because all four threads update the same `totalHits` variable using `totalHits++`. This operation is not thread-safe, so some updates are lost when threads access the

variable at the same time. This is a data race.

Part 2: The Synchronization Trap

- Fix the race condition by adding synchronized to the increment or using AtomicLong.incrementAndGet().
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

Version	Approximate π	Runtime
AtomicLong, 4 threads	3.14180688	908.48 ms
Single thread	3.14183816	472.17 ms

Part 2 Result: After replacing `totalHits++` with `AtomicLong.incrementAndGet()`, the calculated value of π became correct. The 4-thread version produced $\pi = 3.14180688$ and took 908.48 ms. The single-threaded version produced $\pi = 3.14183816$ and took 472.17 ms. The synchronized version was slower because all threads had to update the same shared counter. This creates contention between threads and reduces the benefit of parallel execution.

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.
- Fill out this grid:

Threads	Runtime (ms)	Speedup	Efficiency
1	667.35	1.00x	100.00%
2	397.37	1.68x	83.97%
4	189.59	3.52x	88.00%
8	142.68	4.68x	58.47%
16	82.00	8.14x	50.87%
32	87.67	7.61x	23.79%

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

My CPU has a limited number of physical cores. When I use 16 threads, the threads have to share the available CPU resources. There is also some

overhead from thread scheduling and memory access. Because of this, increasing the number of threads does not give a perfect 2x speedup.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Because all threads were updating the same shared counter. They had to wait for each other to access it safely, which caused synchronization and locking overhead. In my results, the 4-thread `AtomicLong` version took 908.48 ms, while the single-thread version took only 472.17 ms. So synchronization made the multi-threaded version slower.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).